

OOP 練習 1：為什麼要 OOP——第一個類別

學習目標

- 說得出 FORM + 全域變數寫法的限制：狀態沒有保護、想要「第二份狀態」只能再宣告一組變數
- 會寫 local class：CLASS ... DEFINITION (宣告長相) 與 CLASS ... IMPLEMENTATION (實作內容)
- 會宣告屬性 (DATA) 與方法 (METHODS)，理解 PUBLIC SECTION
- 會用 TYPE REF TO 宣告物件參考、NEW 建立物件、-> 存取屬性與方法
- 理解「類別是藍圖，物件是實體」：每個物件有自己的一份屬性

事前準備

建立程式 ZR_0001_<你的姓名縮寫>，套件 \$TMP。

op01–op04 都用 SE38 寫 **local class** (類別跟程式寫在一起)；op05 起改用 ADT (Eclipse) 開發全域類別。

題目需求

1. 定義 local class **lcl_carrier_counter**：
 - 公開屬性 **mv_carrid** (TYPE **s_carr_id**，航空公司代碼)、**mv_flights** (TYPE **i**，累計航班數)
 - 方法 **add_flight** (無參數，計數 +1)、**print** (輸出代碼與計數)
2. 主程式 **NEW** 出兩個物件 (AA、LH 各一)，直接指定各自的 **mv_carrid**
3. AA 的物件呼叫 **add_flight** 三次、LH 的呼叫一次，最後各自 **print**——證明兩個物件的計數互不干擾
4. 回頭想練習 8：同樣「兩家航空公司各自計數」用全域變數 + FORM 要怎麼寫？(每家一個全域變數，第三家、第四家呢？)

預期輸出 (範例)

```
=== 兩個物件，各自的狀態 ===  
AA 累計航班數：      3  
LH 累計航班數：      1
```

團隊實務備註

- 實例屬性的前綴慣例：**mv_ / mt_ / ms_ / mo_** (m = member)，對照風格規範既有的 **lv_** (區域) 與 **gv_** (全域) 系列
- 團隊風格規範要求「新的商業邏輯盡量寫在 Class 的 Method 中」——本課程整個就是在教這件事的原因與做法
- 方法裡存取自己的屬性可以直接寫屬性名 (**me->** 可省略)，團隊慣例是省略

思考題

1. **go_aa** 與 **go_lh** 都來自 **lcl_carrier_counter**，為什麼 **mv_flights** 各記各的？用一句話說清楚「類別」和「物件」的差別。

2. 現在任何地方都能寫 `go_aa->mv_flights = 999`. 直接竄改計數——這樣的設計有什麼風險？（op03 的封裝就是在解這題）
3. 只宣告 `DATA go_x TYPE REF TO lcl_carrier_counter`. 而不 `NEW`，就呼叫 `go_x->add_flight()`. 會發生什麼事？（動手試——這是 OOP 最常見的 runtime error）

答案

見 `zr_oo01_first_class.prog.abap`（SAP 端程式 `ZR_0001_FIRST_CLASS`）。